09. Validator Client 아키텍처 및 서비스 분석

개요

Lighthouse의 Validator Client는 Ethereum 2.0 네트워크에서 검증자(validator) 역할을 수행하는 핵심 컴포넌트입니다. 이 문서에서는 `ProductionValidatorClient` 구조체를 중심으로 한 전체 아키텍처와 각 서비스들의 상호작용, 그리고 초기화 과정을 실제 코드와 함께 상세히 분석합니다.

ProductionValidatorClient 구조체 분석

핵심 구조체 정의

```
// validator_client/src/lib.rs

#[derive(Clone)]

pub  struct ProductionValidatorClient<E: EthSpec> {

context: RuntimeContext<E>,

duties_service: Arc<DutiesService<ValidatorStore<E>, SystemTimeSlotClock>>,

block_service: BlockService<ValidatorStore<E>, SystemTimeSlotClock>,

attestation_service: AttestationService<ValidatorStore<E>, SystemTimeSlotClock>,

sync_committee_service: SyncCommitteeService<ValidatorStore<E>, SystemTimeSlotCl

doppelganger_service: Option<Arc<DoppelgangerService>>,

preparation_service: PreparationService<ValidatorStore<E>, SystemTimeSlotClock>,

validator_store: Arc<ValidatorStore<E>>,

slot_clock: SystemTimeSlotClock,
```

```
http_api_listen_addr: Option<SocketAddr>,

config: Config,

genesis_time: u64,

}
```

주요 컴포넌트 역할

1. RuntimeContext

- **역할**: 실행 환경과 설정 정보를 관리
- **포함 요소**:
- TaskExecutor: 비동기 작업 실행 관리
- ETH2 설정: 네트워크 파라미터와 체인 스펙
- 로깅 컴포넌트: 구조화된 로깅 시스템

2. ValidatorStore

- **역할**: 검증자 키 관리 및 슬래싱 보호
- **핵심 기능**:
- 검증자 키스토어 관리
- BLS 서명 생성 및 검증
- 슬래싱 보호 데이터베이스 연동

3. SlotClock (SystemTimeSlotClock)

- **역할**: 이더리움 2.0 시간 체계 관리
- **기능**:

- 현재 슬롯 계산
- 다음 슬롯까지의 시간 계산
- 에포크 전환 감지

주요 서비스 아키텍처

1. DutiesService - 듀티 관리 서비스

구조체 정의

```
// validator_client/validator_services/src/duties_service.rs
pub struct DutiesService<S: ValidatorStore, T: SlotClock + 'static> {
// 내부 구현은 Inner 구조체에 위임
inner: Arc<Inner<S, T>>,
}

struct Inner<S: ValidatorStore, T: SlotClock + 'static> {
validator_store: Arc<S>,
slot_clock: T,
beacon_nodes: Arc<BeaconNodeFallback<T>>,
spec: ChainSpec,
// 듀티 정보 저장
attester_duties: RwLock<HashMap<Epoch, HashMap<PublicKeyBytes, AttesterData>>>,
proposer_duties: RwLock<HashMap<Epoch, HashMap<PublicKeyBytes, ProposerData>>>,
// ... 기타 필드들
```

```
}
```

핵심 기능

- **검증자 듀티 조회**: `poll_beacon_attesters()`, `poll_beacon_proposers()`
- **선택 증명 생성**: 집계자 역할을 위한 증명 생성
- **구독 관리**: 위원회 서브넷 구독 관리

주요 메서드 분석

```
impl<S: ValidatorStore, T: SlotClock> DutiesService<S, T> {
/// 특정 에포크의 증명자 듀티를 조회하고 업데이트
pub async fn poll_beacon_attesters(&self, epoch: Epoch) -> Result<(), Error<S
// 1. 비콘 노드에서 듀티 정보 요청
let response = self.beacon_nodes
.first_success(|beacon_node| async move {
beacon_node.post_validator_duties_attester(epoch, &validator_indices).await
})
.await?;
// 2. 듀티 정보를 내부 저장소에 업데이트
let mut attester_duties = self.attester_duties.write();
attester_duties.insert(epoch, duties_map);
// 3. 위원회 서브넷 구독 처리
self.update_subscription_params(epoch).await?;
Ok(())
```

```
}

/// 블록 제안자 듀티 조회

pub  async  fn  poll_beacon_proposers(&self, epoch: Epoch) -> Result<(), Error<S

// 비콘 노드에서 제안자 듀티 조회 및 저장

}

}
```

2. AttestationService - 증명 서비스

서비스 빌더 패턴

```
// validator_client/validator_services/src/attestation_service.rs

pub  struct AttestationServiceBuilder<S: ValidatorStore, T: SlotClock + 'static>

duties_service: Option<Arc<DutiesService<S, T>>>,

validator_store: Option<Arc<S>>,

slot_clock: Option<T>,

beacon_nodes: Option<Arc<BeaconNodeFallback<T>>>,

executor: Option<TaskExecutor>,

chain_spec: Option<Arc<ChainSpec>>,

disable: bool,

}
```

핵심 동작 흐름

1. **듀티 확인**: DutiesService에서 증명 듀티 조회
2. **증명 데이터 요청**: 비콘 노드에서 AttestationData 획득
3. **서명 생성**: ValidatorStore를 통한 BLS 서명
4. **증명 제출**: 비콘 노드로 서명된 증명 전송

증명 생성 과정

```
impl<S: ValidatorStore, T: SlotClock> AttestationService<S, T> {
/// 증명을 생성하고 제출하는 메인 함수
pub  async  fn  produce_and_publish_attestations(&self, slot: Slot) -> Result<()
// 1. 해당 슬롯의 듀티 조회
let  duties = self.duties_service.get_attestation_duties(slot).await?;
// 2. 각 듀티에 대해 병렬로 증명 처리
let  futures = duties.into_iter().map(|duty| {
self.produce_attestation(duty)
});
join_all(futures).await;
Ok(())
}
async  fn  produce_attestation(&self, duty: DutyAndProof) -> Result<(), Error> {
// 1. 증명 데이터 요청
let  attestation_data = self.beacon_nodes
.first_success(|beacon_node| async  move {
beacon_node.get_validator_attestation_data(
```

```
duty.slot,
duty.committee_index
).await
})
.await?;
// 2. 서명 생성
let signature = self.validator_store
.sign_attestation(duty.pubkey, attestation_data.clone())
.await?;
// 3. 증명 구성 및 제출
let attestation = Attestation {
aggregation_bits: duty.aggregation_bits,
data: attestation_data,
signature,
};
self.beacon_nodes
.first_success(|beacon_node| async move {
beacon_node.post_beacon_pool_attestations(&[attestation.clone()]).await
})
.await?;
Ok(())
}
```

```
}
```

3. BlockService - 블록 제안 서비스

서비스 구조

```rust
// validator_client/validator_services/src/block_service.rs

pub struct BlockServiceBuilder<S, T> {

validator_store: Option<Arc<S>>,

slot_clock: Option<Arc<T>>,

beacon_nodes: Option<Arc<BeaconNodeFallback<T>>>,

proposer_nodes: Option<Arc<BeaconNodeFallback<T>>>, // 별도의 제안자 노드

executor: Option<TaskExecutor>,

chain_spec: Option<Arc<ChainSpec>>,

graffiti: Option<Graffiti>,

graffiti_file: Option<GraffitiFile>,

}
```

블록 제안 과정

1. **제안자 확인**: 현재 슬롯의 블록 제안자인지 확인
2. **블록 생성**: 비콘 노드에서 미서명 블록 요청
3. **블록 서명**: ValidatorStore를 통한 블록 서명
4. **블록 제출**: 서명된 블록을 네트워크에 브로드캐스트

4. SyncCommitteeService - 동기화 위원회 서비스

서비스 구조

```rust
// validator_client/validator_services/src/sync_committee_service.rs
pub struct SyncCommitteeService<S: ValidatorStore, T: SlotClock + 'static> {
inner: Arc<Inner<S, T>>,
}

pub struct Inner<S: ValidatorStore, T: SlotClock + 'static> {
duties_service: Arc<DutiesService<S, T>>,
validator_store: Arc<S>,
slot_clock: T,
beacon_nodes: Arc<BeaconNodeFallback<T>>,
executor: TaskExecutor,
first_subscription_done: AtomicBool,
}
```

동기화 위원회 역할

- **동기화 메시지 생성**: 각 슬롯마다 동기화 메시지 서명
- **집계 참여**: 동기화 위원회 집계자로 선정시 집계 수행
- **서브넷 구독**: 동기화 위원회 서브넷 구독 관리

5. PreparationService - 준비 서비스

서비스 역할

```rust
// validator_client/validator_services/src/preparation_service.rs

pub struct PreparationService<S: ValidatorStore, T: SlotClock + 'static> {

inner: Arc<Inner<S, T>>,

}
```

주요 기능

- **제안자 준비**: 블록 제안을 위한 사전 준비 작업
- **검증자 등록**: MEV-Boost 빌더에 검증자 등록
- **수수료 수신자 설정**: 블록 보상 수신 주소 관리

6. DoppelgangerService - 중복 검증자 보호 서비스

보호 메커니즘

```rust
// validator_client/doppelganger_service/src/lib.rs

#[derive(Debug, PartialEq)]

pub struct DoppelgangerState {

/// 다음 검사할 에포크

next_check_epoch: Epoch,

/// 남은 검사 에포크 수

remaining_epochs: u64,

}


impl DoppelgangerState {
```

```rust
/// 추가 검사가 필요한지 확인
fn requires_further_checks(&self) -> bool {
self.remaining_epochs > 0
}
/// 에포크 검사 완료 처리
fn complete_detection_in_epoch(&mut self, epoch: Epoch) {
self.remaining_epochs = self.remaining_epochs.saturating_sub(1);
self.next_check_epoch = epoch.saturating_add(1_u64);
}
}
```

동작 원리

1. **활성도 모니터링**: 비콘 노드에서 검증자 활성도 조회
2. **중복 감지**: 동일한 검증자가 이미 활동 중인지 확인
3. **보호 조치**: 중복 검증자 발견시 클라이언트 종료

서비스 간 상호작용

의존성 그래프

```
DutiesService (핵심 허브)
↓ 듀티 정보 제공
├── AttestationService
├── BlockService
```

```
├── SyncCommitteeService
└── PreparationService

ValidatorStore (공통 의존성)
↑ 모든 서비스에서 사용
├── 키 관리
├── 서명 생성
└── 슬래싱 보호

BeaconNodeFallback (네트워크 통신)
↑ 모든 서비스에서 사용
├── API 호출
├── 데이터 조회
└── 메시지 제출
```

메시지 흐름

1. **DutiesService**: 주기적으로 듀티 정보 업데이트
2. **BlockService**: 블록 제안 알림 수신 → 블록 생성 및 제출
3. **AttestationService**: 슬롯 시작 → 증명 생성 및 제출
4. **SyncCommitteeService**: 동기화 듀티 확인 → 동기화 메시지 생성

초기화 과정 분석

1. 설정 및 환경 초기화

```
impl<E: EthSpec> ProductionValidatorClient<E> {
pub  async  fn  new(context: RuntimeContext<E>, config: Config) -> Result<Self,
// 1. 파일 디스크립터 한계 증가
fdlimit::raise_fd_limit()?;
// 2. 메트릭 서버 시작 (선택적)
let  validator_metrics_ctx = if  config.http_metrics.enabled {
// 메트릭 서버 초기화 및 시작
};
// 3. 모니터링 클라이언트 시작 (선택적)
if  let Some(monitoring_config) = &config.monitoring_api {
let  monitoring_client = MonitoringHttpClient::new(monitoring_config)?;
monitoring_client.auto_update(context.executor.clone(), ...);
};
```

2. 검증자 정의 및 키스토어 로딩

```
// 4. 검증자 정의 파일 로딩
let  mut  validator_defs = ValidatorDefinitions::open_or_create(&config.validato
// 5. 자동 키스토어 발견 (선택적)
if !config.disable_auto_discover {
let  new_validators = validator_defs
.discover_local_keystores(&config.validator_dir, &config.secrets_dir)?;
```

```
validator_defs.save(&config.validator_dir)?;
}
// 6. 검증자 초기화
let validators = InitializedValidators::from_definitions(
validator_defs,
config.validator_dir.clone(),
config.initialized_validators.clone(),
).await?;
```

3. 슬래싱 보호 초기화

```
// 7. 슬래싱 보호 데이터베이스 초기화
let slashing_db_path = config.validator_dir.join(SLASHING_PROTECTION_FILENAME);
let slashing_protection = if config.init_slashing_protection || voting_pubkeys
SlashingDatabase::open_or_create(&slashing_db_path)?
} else {
SlashingDatabase::open(&slashing_db_path)?
};
// 8. 검증자 등록 확인
if config.init_slashing_protection {
slashing_protection.register_validators(voting_pubkeys.iter().copied())?;
} else {
slashing_protection.check_validator_registrations(voting_pubkeys.iter().copied()
```

```
}
```

4. 비콘 노드 연결 설정

```
// 9. 비콘 노드 HTTP 클라이언트 설정
let beacon_node_setup = |x: (usize, &SensitiveUrl)| {
let mut beacon_node_http_client_builder = ClientBuilder::new();
// TLS 인증서 추가 (선택적)
if let Some(certificates) = &config.beacon_nodes_tls_certs {
for cert in certificates {
beacon_node_http_client_builder = beacon_node_http_client_builder
.add_root_certificate(load_pem_certificate(cert)?);
}
}
// 타임아웃 설정
let timeouts = if i < last_beacon_node_index && !config.use_long_timeouts {
Timeouts::use_optimized_timeouts(slot_duration)
} else {
Timeouts::set_all(slot_duration.saturating_mul(config.long_timeouts_multiplier))
};
Ok(BeaconNodeHttpClient::from_components(url.clone(), client, timeouts))
};
```

5. 제네시스 정보 획득

```
// 10. 비콘 노드에서 제네시스 정보 획득
let (genesis_time, genesis_validators_root) = tokio::select! {
tuple = init_from_beacon_node::<E>(&beacon_nodes, &proposer_nodes) => tuple?,
() = context.executor.exit() => return Err("Shutting down".to_string())
};
// 11. 슬롯 클록 초기화
let  slot_clock = SystemTimeSlotClock::new(
context.eth2_config.spec.genesis_slot,
Duration::from_secs(genesis_time),
Duration::from_secs(context.eth2_config.spec.seconds_per_slot),
);
```

6. 서비스 초기화

```
// 12. ValidatorStore 생성
let  validator_store = Arc::new(LighthouseValidatorStore::new(
validators,
slashing_protection,
genesis_validators_root,
context.eth2_config.spec.clone(),
doppelganger_service.clone(),
```

```
slot_clock.clone(),
&config.validator_store,
context.executor.clone(),
));
// 13. DutiesService 생성
let  duties_service = Arc::new(
DutiesServiceBuilder::new()
.slot_clock(slot_clock.clone())
.beacon_nodes(beacon_nodes.clone())
.validator_store(validator_store.clone())
.spec(context.eth2_config.spec.clone())
.executor(context.executor.clone())
.build()?
);
// 14. 기타 서비스들 생성
let  block_service = BlockServiceBuilder::new()
.slot_clock(slot_clock.clone())
.validator_store(validator_store.clone())
.beacon_nodes(beacon_nodes.clone())
.build()?;
let  attestation_service = AttestationServiceBuilder::new()
.duties_service(duties_service.clone())
.validator_store(validator_store.clone())
```

```
.build()?;
```

서비스 시작 과정

1. HTTP API 서버 시작

```
impl<E: EthSpec> ProductionValidatorClient<E> {
pub  async  fn  start_service(&mut  self) -> Result<(), String> {
// 1. 블록 서비스 알림 채널 생성
let (block_service_tx, block_service_rx) = mpsc::channel(channel_capacity);
// 2. HTTP API 서버 시작 (선택적)
self.http_api_listen_addr = if  self.config.http_api.enabled {
let  ctx = Arc::new(validator_http_api::Context { ... });
let (listen_addr, server) = validator_http_api::serve::<_, E>(ctx, exit)?;
self.context.executor.spawn_without_exit(server, "http-api");
Some(listen_addr)
} else {
None
};
```

2. 제네시스 대기

```
// 3. 제네시스 시간까지 대기
```

```
wait_for_genesis(self.genesis_time).await?;
```

3. 서비스 시작

```
// 4. DutiesService 업데이트 서비스 시작
duties_service::start_update_service(self.duties_service.clone(), block_service_
// 5. BlockService 시작
self.block_service
.clone()
.start_update_service(block_service_rx)?;
// 6. AttestationService 시작
self.attestation_service
.clone()
.start_update_service(&self.context.eth2_config.spec)?;
// 7. SyncCommitteeService 시작
self.sync_committee_service
.clone()
.start_update_service(&self.context.eth2_config.spec)?;
// 8. PreparationService 시작
self.preparation_service
.clone()
.start_update_service(&self.context.eth2_config.spec)?;
```

4. 보조 서비스 시작

```
// 9. DoppelgangerService 시작 (선택적)
if  let Some(doppelganger_service) = self.doppelganger_service.clone() {
DoppelgangerService::start_update_service(
doppelganger_service,
self.context.service_context(DOPPELGANGER_SERVICE_NAME.into()),
self.validator_store.clone(),
self.duties_service.beacon_nodes.clone(),
self.duties_service.slot_clock.clone(),
)?
}
// 10. 알림 서비스 시작
spawn_notifier(
self.duties_service.clone(),
context.executor,
&self.context.eth2_config.spec,
)?;
// 11. 지연시간 측정 서비스 시작 (선택적)
if  self.config.enable_latency_measurement_service {
latency_service::start_latency_service(
self.context.executor.clone(),
self.duties_service.slot_clock.clone(),
```

```
self.duties_service.beacon_nodes.clone(),
);
}
```

에러 처리 및 복구 메커니즘

1. 네트워크 장애 처리

- **BeaconNodeFallback**: 여러 비콘 노드 중 사용 가능한 노드로 자동 전환
- **재시도 로직**: 일시적 네트워크 오류에 대한 지수 백오프 재시도
- **타임아웃 관리**: 상황에 따른 적응적 타임아웃 설정

2. 검증자 보호 메커니즘

- **슬래싱 보호**: 데이터베이스 기반 중복 서명 방지
- **Doppelganger 보호**: 중복 검증자 실행 감지 및 차단
- **키 관리**: 안전한 키스토어 로딩 및 서명 과정

3. 상태 복구

- **듀티 재동기화**: 네트워크 복구 후 듀티 정보 재획득
- **메트릭 복원**: 서비스 재시작 후 메트릭 상태 복원
- **구독 재설정**: 위원회 서브넷 구독 재설정

성능 최적화

1. 병렬 처리

- **증명 생성**: 여러 검증자의 증명을 병렬로 처리
- **듀티 조회**: 배치 요청을 통한 효율적인 듀티 조회
- **서명 작업**: 비동기 서명 처리로 블로킹 최소화

2. 캐싱 전략

- **듀티 캐싱**: 에포크별 듀티 정보 캐싱
- **선택 증명 캐싱**: 집계자 선택 증명 사전 계산
- **네트워크 응답 캐싱**: 반복적인 API 호출 최적화

3. 메모리 관리

- **히스토리 정리**: 오래된 듀티 정보 자동 정리
- **슬래싱 DB 정리**: 주기적인 슬래싱 보호 DB 정리
- **메트릭 최적화**: 대규모 검증자 환경에서의 메트릭 관리

결론

Lighthouse의 Validator Client는 모듈화된 서비스 아키텍처를 통해 복잡한 검증자 역할을 효율적으로 수행합니다. `ProductionValidatorClient` 를 중심으로 한 각 서비스들은 명확한 책임 분리와 강력한 상호작용을 통해 안정적이고 확장 가능한 검증자 클라이언트를 구현합니다.

특히 DutiesService를 허브로 하는 서비스 간 협력, 포괄적인 에러 처리 및 복구 메커니즘, 그리고 성능 최적화를 위한 다양한 전략들이 실제 프로덕션 환경에서의 안정적인 검증자 운영을 가능하게 합니다.